🎮 Solana Esports Platform

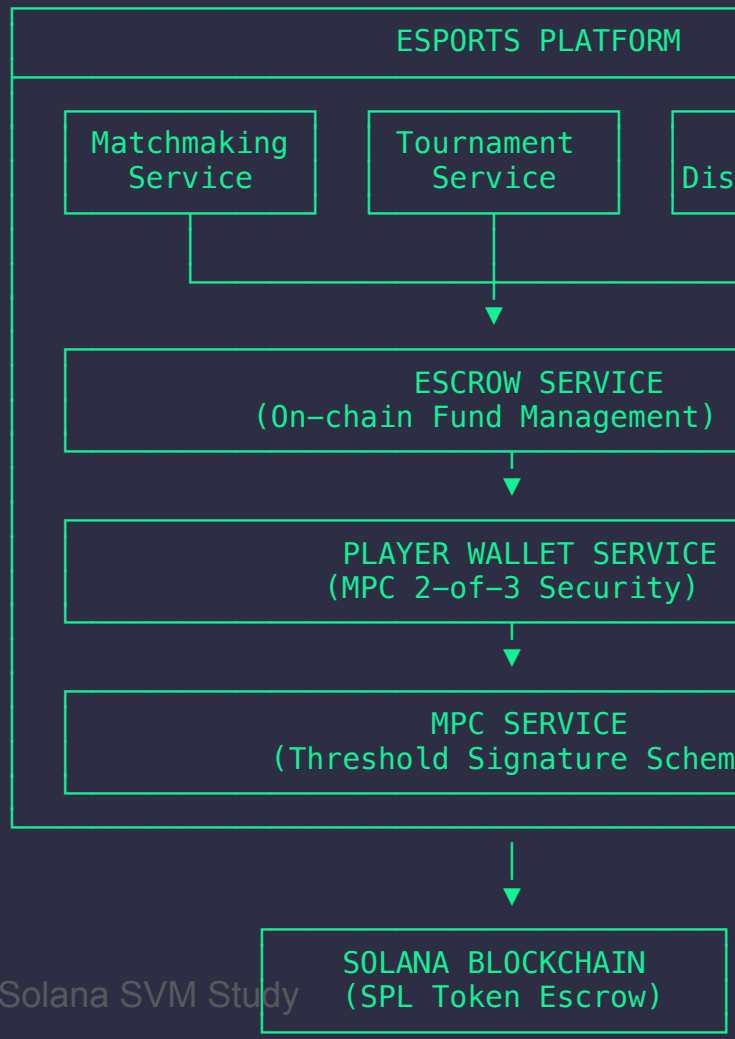
Monetized Matchmaking, MPC Wallets & Prize Distribution

Production-Grade Implementation

📋 Agenda

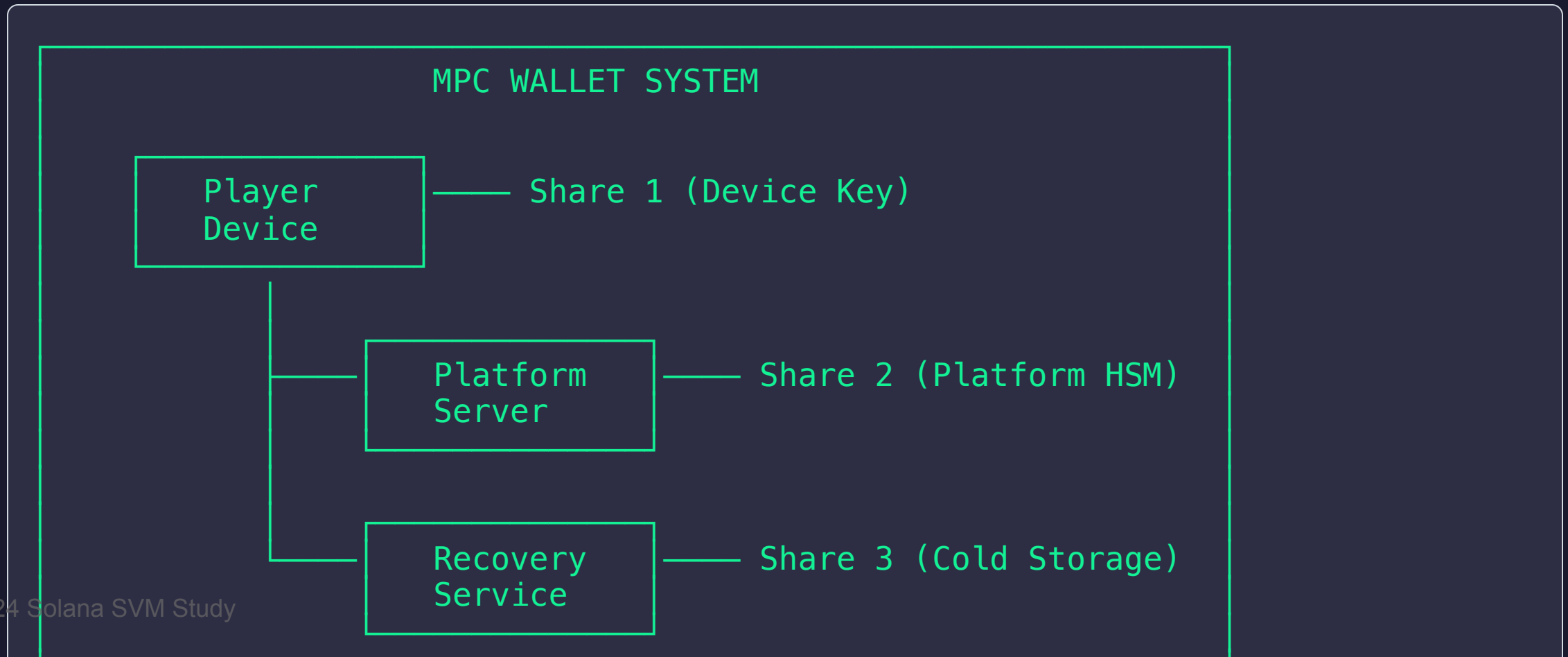
1. **System Overview** - Architecture & Components
2. **MPC Wallet Integration** - Secure Player Wallets
3. **Matchmaking System** - Entry Fees & Escrow
4. **Prize Distribution** - Automated Payouts
5. **Tournament Management** - Bracket Generation
6. **Security Practices** - Production Considerations
7. **Implementation Details** - Code Walkthrough

System Architecture Overview



🔐 MPC Wallet Architecture

Multi-Party Computation (2-of-3 Threshold)



🔐 MPC Benefits for Gaming

Feature	Traditional Wallet	MPC Wallet
Single Point of Failure	❌ Yes	✅ No
Key Recovery	❌ Complex	✅ Built-in
Account Takeover Risk	❌ High	✅ Mitigated
User Experience	❌ Seed Phrases	✅ Seamless
Platform Control	❌ None	✅ Fraud Prevention

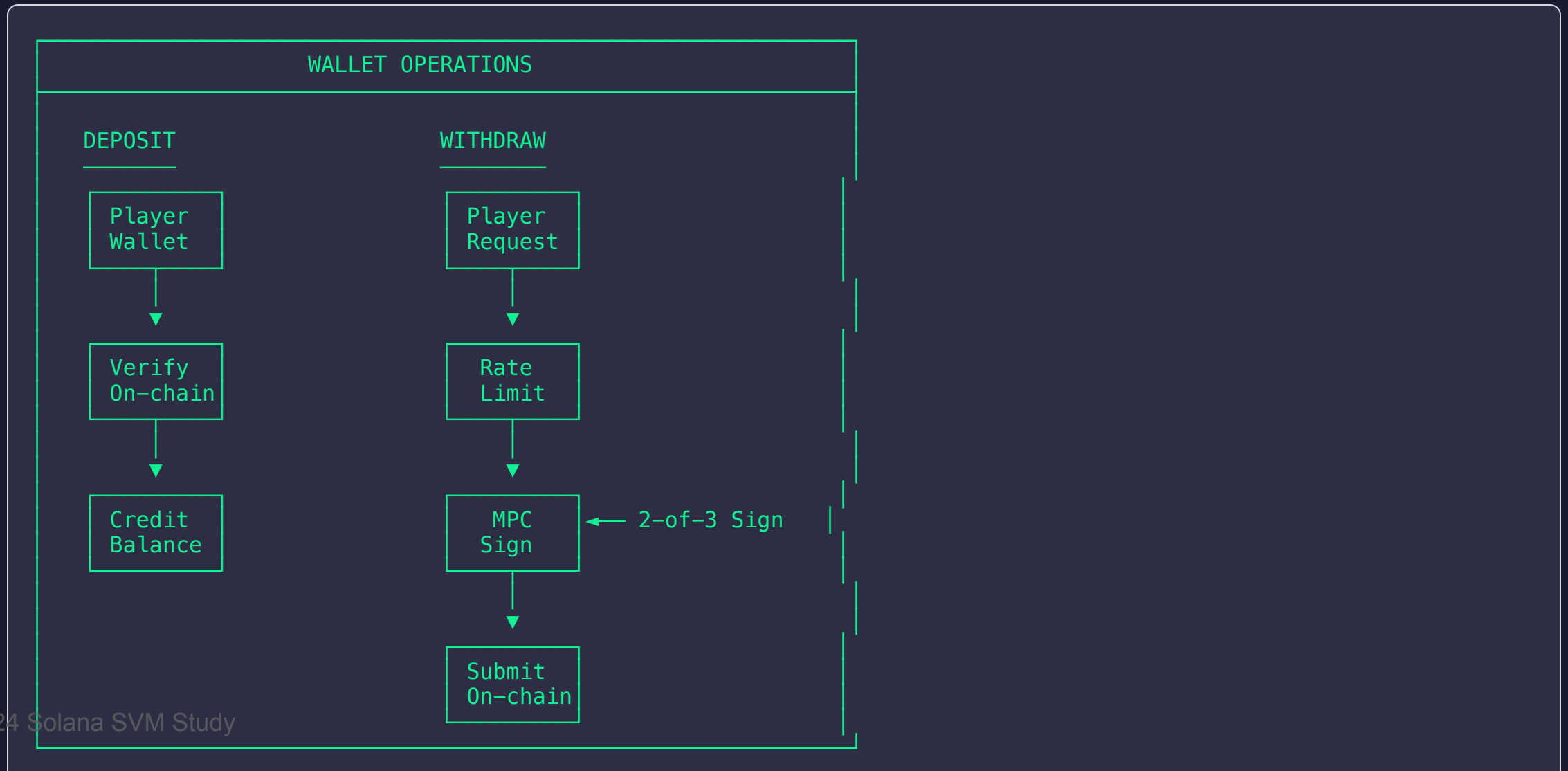
Key Advantages:

- **No seed phrase exposure** to players
- **Platform co-signing** prevents fraud
- **Recovery service** for lost devices
- **Rate limiting** on withdrawals

💰 Player Wallet Entity

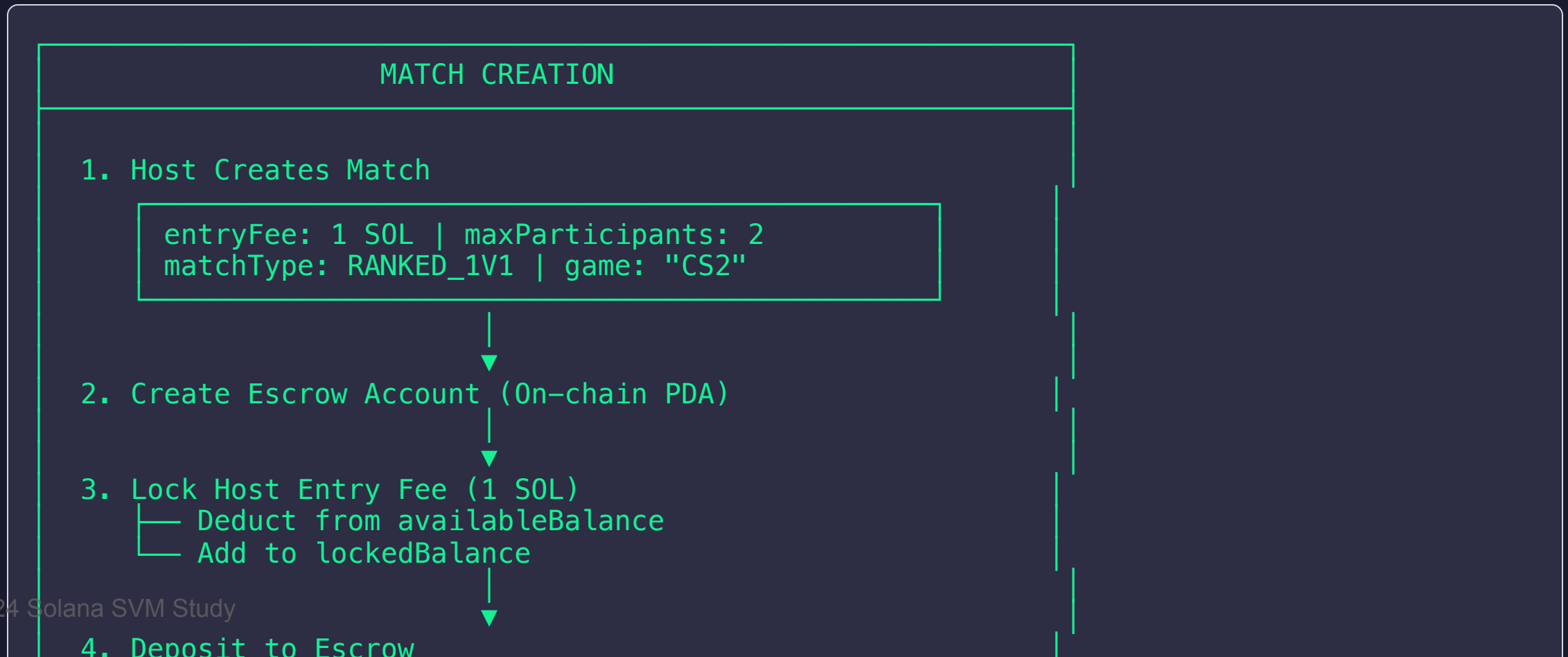
```typescript
@Entity('player_wallets')
export class PlayerWallet {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  playerId: string;

  @Column()
  mpcWalletId: string;  // Link to MPC system

  @Column()
  publicKey: string;    // Solana address

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  availableBalance: string;  // Lamports

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  lockedBalance: string;     // In escrow

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalLimit: string;

  @Column({ type: 'enum', enum: PlayerWalletStatus })
  status: PlayerWalletStatus;
}
```

🔒 Wallet Operations Flow

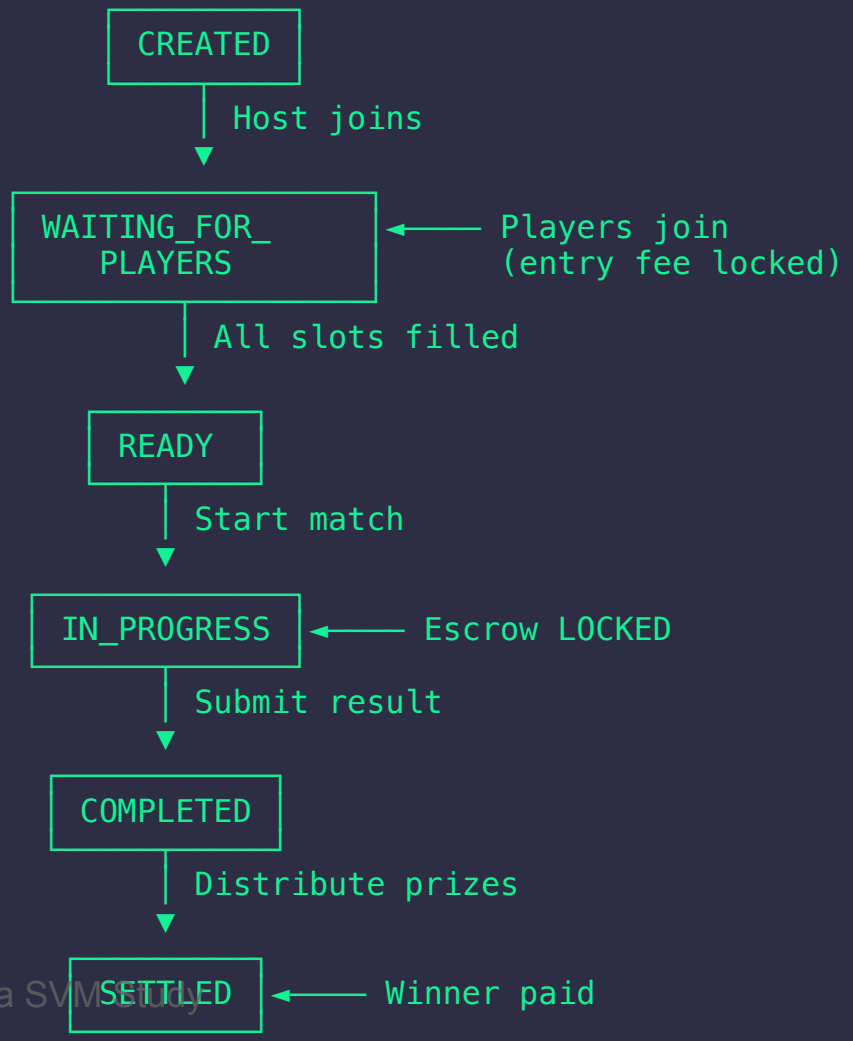


🎯 Matchmaking System

Entry Fee & Escrow Flow



🎮 Match Lifecycle



💵 Prize Distribution Logic

Winner Takes Pool (minus platform fee)

```
async distributeMatchPrizes(matchId: string) {
  const match = await this.matchRepository.findOne({
    where: { id: matchId },
    relations: ['escrow', 'participants'],
  });

  // Calculate prize pool
  const totalPool = BigInt(match.entryFee) *
                    BigInt(match.participants.length);

  // Deduct platform fee (e.g., 5%)
  const platformFee = totalPool * BigInt(match.platformFeePercentage) / 100n;
  const winnerPrize = totalPool - platformFee;

  // Credit winner's wallet
  await this.walletService.creditPrize(
    match.winnerId,
    winnerPrize.toString(),
    matchId
  );
```

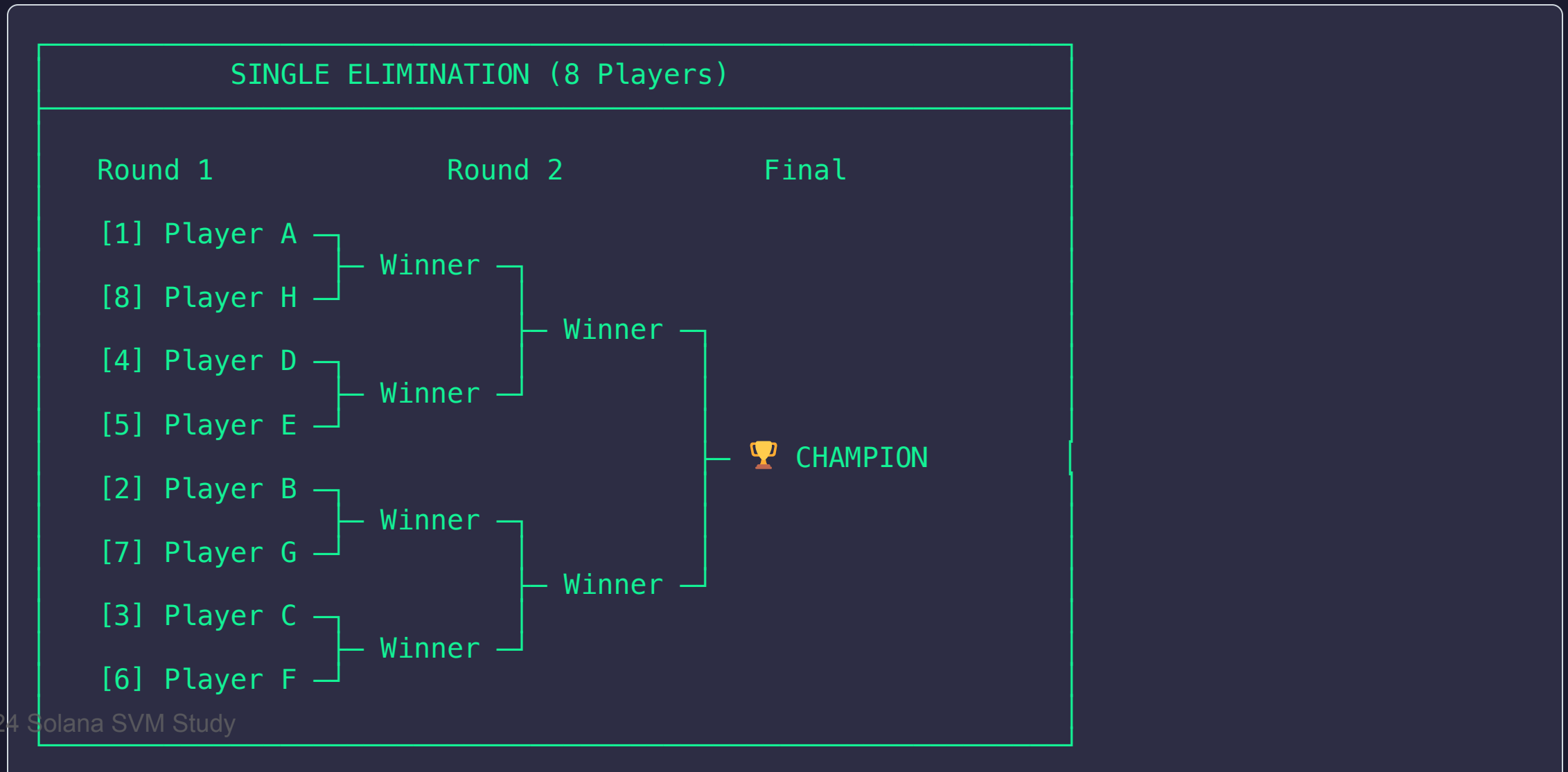
📊 Prize Distribution Flow

🏆 Tournament System

Bracket Generation

```
async generateBracket(tournamentId: string): Promise<Tournament> {
  const tournament = await this.tournamentRepository.findOne(/*...*/);
  const registrations = await this.registrationRepository.find({
    where: { tournamentId },
    order: { seed: 'ASC' },
  });

  // Single elimination: seeded matchups
  // 1 vs 8, 2 vs 7, 3 vs 6, 4 vs 5
  const bracket = this.createSingleEliminationBracket(
    registrations,
    tournament.maxParticipants
  );

  // Create match entities for each bracket slot
  const matches = await this.createBracketMatches(
    tournament,
    bracket
  );

  return this.tournamentRepository.save({
    tournament,
```

Tournament Bracket Structure



💰 Tournament Prize Structure

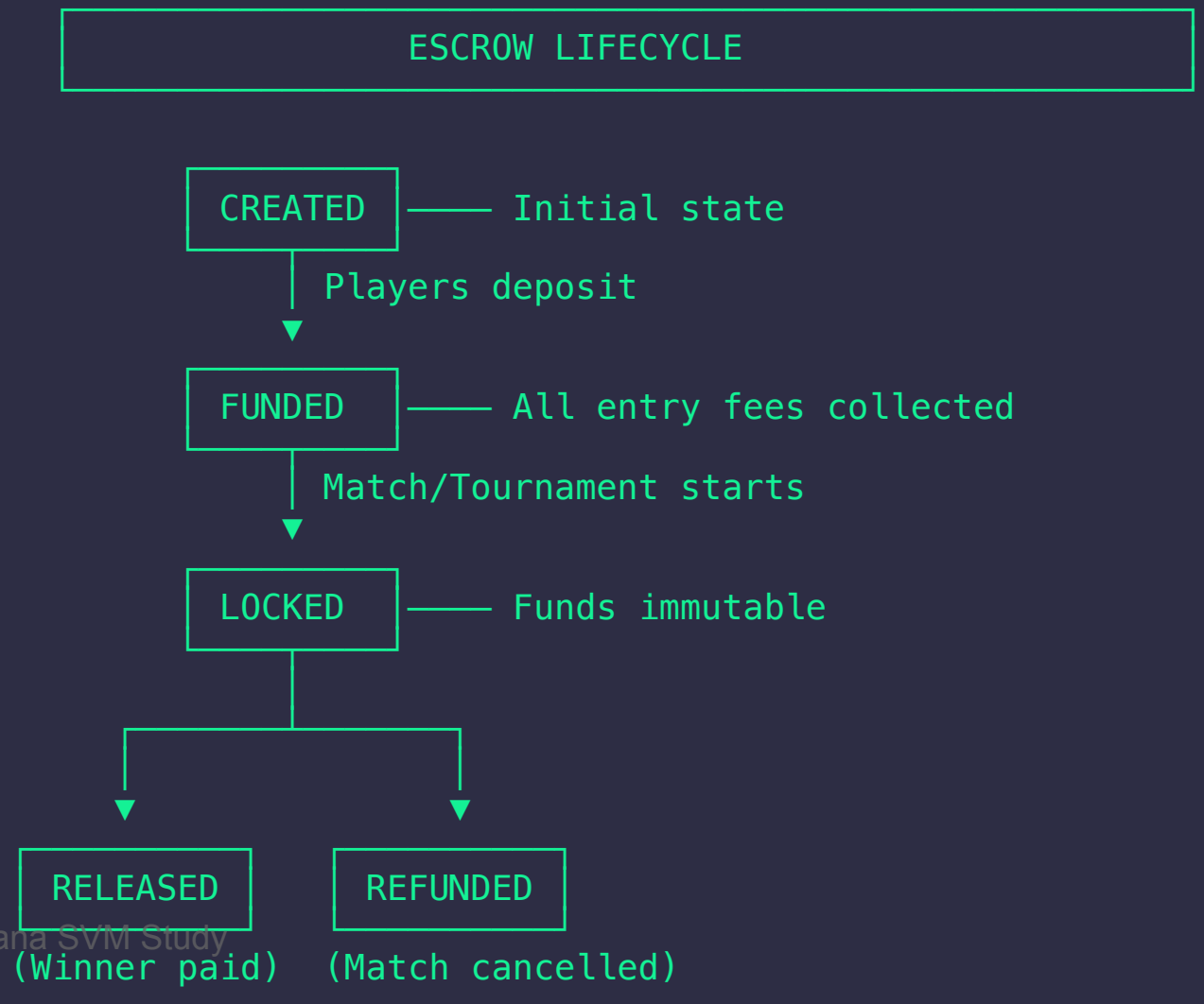
```
const prizeStructure = {
  1: 50,  // 1st place: 50% of pool
  2: 30,  // 2nd place: 30% of pool
  3: 20,  // 3rd/4th: 20% split
};

// Example: 16 players x 10 SOL entry = 160 SOL pool
// Platform fee (5%): 8 SOL
// Distributable: 152 SOL

// Payouts:
// 1st: 76 SOL
// 2nd: 45.6 SOL
// 3rd: 30.4 SOL
```

Placement	Percentage	Prize (SOL)
🥇 1st	50%	76.0

🔐 Escrow Account Entity

```typescript
@Entity('escrow_accounts')
export class EscrowAccount {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'enum', enum: EscrowType })
  type: EscrowType;  // MATCH | TOURNAMENT

  @Column()
  reference: string;  // match_id or tournament_id

  @Column()
  escrowPda: string;  // Program Derived Address

  @Column({ type: 'decimal', precision: 20, scale: 0 })
  amount: string;

  @Column({ type: 'enum', enum: EscrowStatus })
  status: EscrowStatus;
  // CREATED → FUNDED → LOCKED → RELEASED/REFUNDED
}
```

Escrow State Machine



🛡️ Security Measures

Production Considerations

Wallet Security

- ✅ MPC 2-of-3 threshold signatures
- ✅ Daily withdrawal limits
- ✅ Rate limiting on transactions
- ✅ Account suspension capabilities

Escrow Security

- ✅ On-chain PDA escrow accounts
- ✅ Atomic state transitions
- ✅ Immutable locked funds

🔐 Rate Limiting Implementation

```
@Entity('player_wallets')
export class PlayerWallet {
  @Column({ type: 'decimal', precision: 20, scale: 0 })
  dailyWithdrawalLimit: string;

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalAmount: string;

  @Column({ type: 'timestamp', nullable: true })
  dailyWithdrawalResetAt: Date;

  canWithdraw(amount: bigint): boolean {
    if (this.status !== PlayerWalletStatus.ACTIVE) return false;

    const available = BigInt(this.availableBalance);
    if (amount > available) return false;

    // Check daily limit
    const dailyUsed = BigInt(this.dailyWithdrawalAmount);
    const dailyLimit = BigInt(this.dailyWithdrawalLimit);

    return (dailyUsed + amount) <= dailyLimit;
  }
}
```

📝 Transaction Types

```
export enum WalletTransactionType {
  // Inflows
  DEPOSIT = 'DEPOSIT',              // External deposit
  PRIZE_WIN = 'PRIZE_WIN',          // Match/tournament win
  REFUND = 'REFUND',                // Cancelled match refund

  // Outflows
  WITHDRAWAL = 'WITHDRAWAL',        // External withdrawal
  ENTRY_FEE = 'ENTRY_FEE',          // Match/tournament entry

  // Internal
  LOCK = 'LOCK',                    // Lock for escrow
  UNLOCK = 'UNLOCK',                // Release from escrow
}
```

Full Audit Trail

Every transaction logged with signatures and timestamps

🔄 Complete Match Flow

```
                    COMPLETE MATCH FLOW

1. CREATE MATCH
   └── Host deposits entry fee → Escrow created

2. JOIN MATCH
   └── Player deposits entry fee → Added to escrow

3. START MATCH
   └── Escrow LOCKED → Players notified

4. PLAY GAME
   └── External game server handles gameplay

5. SUBMIT RESULT
   └── Winner declared → Match COMPLETED

6. DISTRIBUTE PRIZES
   └── Winner wallet credited → Escrow RELEASED

7. SETTLEMENT
   └── All balances updated → Match SETTLED
```

🚀 API Endpoints

Matchmaking

Method	Endpoint	Description
POST	`/api/matches`	Create match
POST	`/api/matches/:id/join`	Join match
POST	`/api/matches/:id/start`	Start match
POST	`/api/matches/:id/result`	Submit result
GET	`/api/matches/open`	List open matches

Wallets

Method	Endpoint	Description
POST	`/api/wallets`	Create MPC wallet

🚀 API Endpoints (cont.)

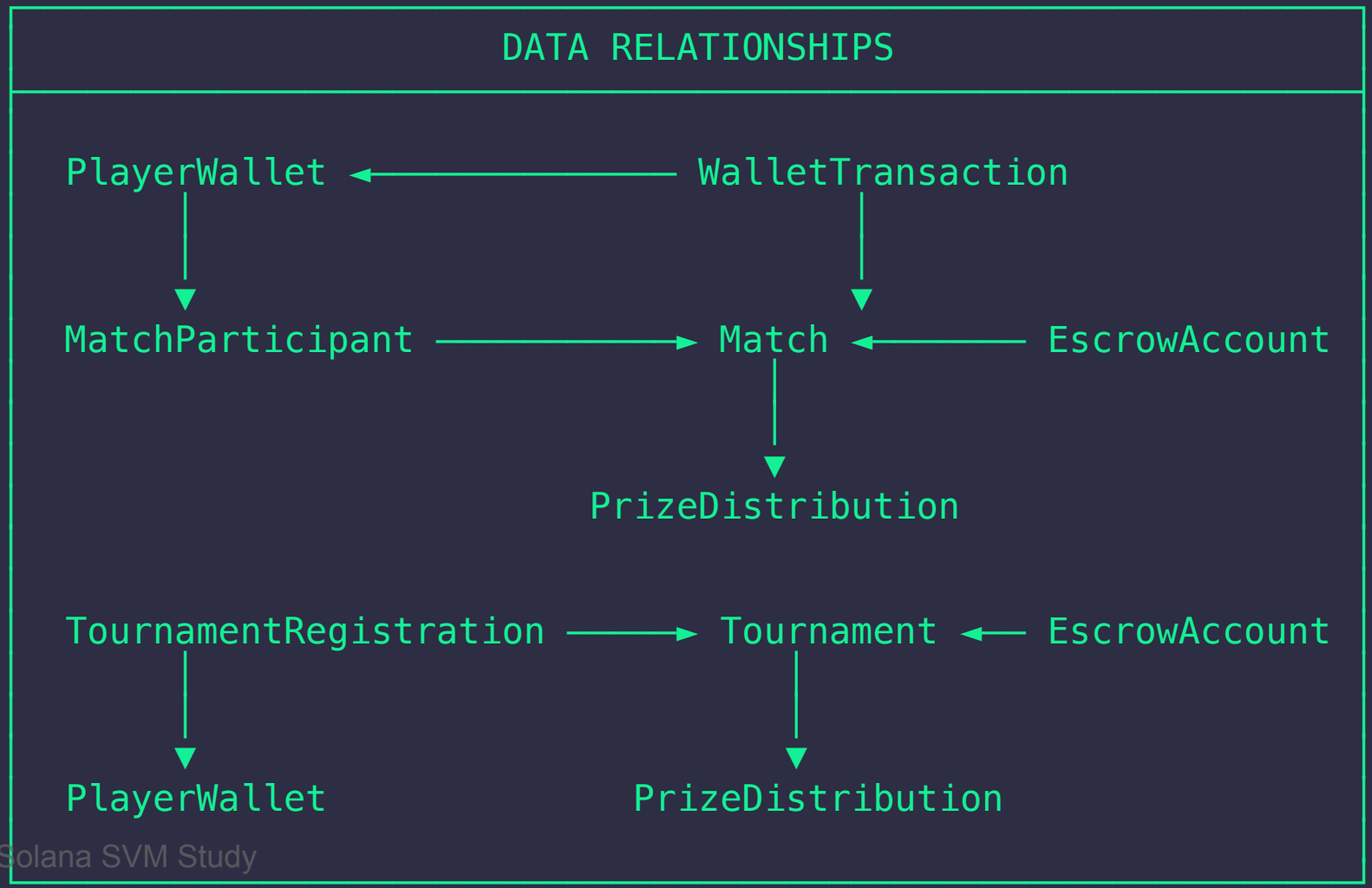
Tournaments

Method	Endpoint	Description
POST	`/api/tournaments`	Create tournament
POST	`/api/tournaments/:id/register`	Register player
POST	`/api/tournaments/:id/bracket`	Generate bracket
POST	`/api/tournaments/:id/start`	Start tournament
GET	`/api/tournaments/:id/leaderboard`	Get standings

Prizes

Method	Endpoint	Description
POST	`/api/prizes/match/:id/distribute`	Pay match prizes

📊 Data Model Overview



🧪 Testing Strategy

Unit Tests

```
describe('EscrowService', () => {
  it('should create escrow for match', async () => {
    const result = await escrowService.createEscrow({
      type: EscrowType.MATCH,
      reference: 'match_123',
      tokenMint: SOL_MINT,
    });

    expect(result.status).toBe(EscrowStatus.CREATED);
    expect(result.escrowPda).toBeDefined();
  });

  it('should reject release for unlocked escrow', async () => {
    await expect(
      escrowService.releaseEscrow('escrow_unlocked')
    ).rejects.toThrow(BadRequestException);
```

🧪 Integration Testing

```
describe('Match E2E', () => {
  it('should complete full match flow', async () => {
    // 1. Create wallets for both players
    const wallet1 = await walletService.createWallet({ playerId: 'p1' });
    const wallet2 = await walletService.createWallet({ playerId: 'p2' });

    // 2. Deposit funds
    await walletService.deposit({ playerId: 'p1', amount: '2000000000' });
    await walletService.deposit({ playerId: 'p2', amount: '2000000000' });

    // 3. Create and join match
    const match = await matchService.createMatch({
      hostPlayerId: 'p1',
      entryFee: '1000000000',
    });
    await matchService.joinMatch(match.id, 'p2');

    // 4. Start and complete
    await matchService.startMatch(match.id);
    await matchService.submitResult(match.id, { winnerId: 'p1' });

    // 5. Distribute prizes
    await prizeService.distributeMatchPrizes(match.id);

    // 6. Verify balances
    const balance1 = await walletService.getBalance('p1');
    expect(balance1.availableBalance).toBe('2900000000'); // Won
  });
});
```

📈 Monitoring & Observability

Key Metrics to Track

```
// Custom metrics for prize distribution
@Injectable()
export class PrizeDistributionService {
  private readonly prizeDistributedCounter = new Counter({
    name: 'esports_prizes_distributed_total',
    help: 'Total prizes distributed',
    labelNames: ['type', 'status'],
  });

  private readonly prizeAmountGauge = new Gauge({
    name: 'esports_prize_amount_sol',
    help: 'Prize amount in SOL',
    labelNames: ['type'],
  });

  async distributePrize(/*...*/) {
    // ... distribution logic
    this.prizeDistributedCounter.inc({ type: 'match', status: 'success' });
```

📈 Key Metrics Dashboard

Metric	Description	Alert Threshold
`escrow_balance_total`	Total SOL in escrow	> 1000 SOL
`match_completion_rate`	% matches completed	< 90%
`prize_distribution_latency`	Time to distribute	> 30s
`mpc_signing_failures`	Failed MPC signs	> 0
`withdrawal_rate_limit_hits`	Rate limit triggers	> 100/hr

Alerts

- 🚨 Escrow balance mismatch
- 🚨 MPC signing failure rate > 1%
- 🚨 Prize distribution backlog
- 🚨 Suspicious withdrawal patterns

🎯 Best Practices Summary

1. Security First

- Always use MPC for player wallets
- Lock escrow before match starts
- Implement withdrawal limits

2. Atomic Operations

- Use database transactions
- Handle partial failures gracefully
- Implement idempotency keys

3. Auditability

- Log all financial transactions

🔮 Future Enhancements

Roadmap Items

1. **Cross-Game Wallet**
   - Single wallet across multiple games
   - Unified balance management
2. **NFT Integration**
   - Tournament trophy NFTs
   - Achievement badges
3. **DAO Governance**
   - Community-driven prize pools
   - Voting on platform fees

📚 Resources

Documentation

- Solana Web3.js Docs
- SPL Token Program
- MPC Cryptography

Code References

- `/src/modules/esports/` - Full implementation
- `/docs/diagrams/16-esports-matchmaking.md` - Architecture docs
- `/src/modules/mpc/` - MPC wallet integration

Tools

- Anchor Framework for Solana programs

? Questions?

Key Takeaways

1. **MPC wallets** provide security without UX friction
2. **Escrow accounts** ensure trustless prize distribution
3. **Atomic state machines** prevent fund loss
4. **Platform co-signing** enables fraud prevention
5. **Full audit trails** support dispute resolution

🙏 Thank You!

Contact & Resources

- 📧 Project Repository: `github.com/psavelis/solana-svm-study`
- 📖 Documentation: `/docs/diagrams/16-esports-matchmaking.md`
- 🧪 Tests: `/src/modules/esports/__tests__/`

🎮 Let's Build!

Production-Ready Esports on Solana